

Laboratorio 2 - Data Science

Pivi Riccardo

Ottobre 2025

1 Punto 1: Creare i Dati

Sono stati creati 200 dati (x,y) nel modo seguente:

```
1 N = 200
2
3 X = np.random.rand(N,1)
4 Y = np.sin(2*np.pi*X) + np.random.uniform(-0.2,0.2, size=(N,1))
```

Si riporta il grafico [Fig.1] del confronto dei dati con la funzione $y = \sin 2\pi x$

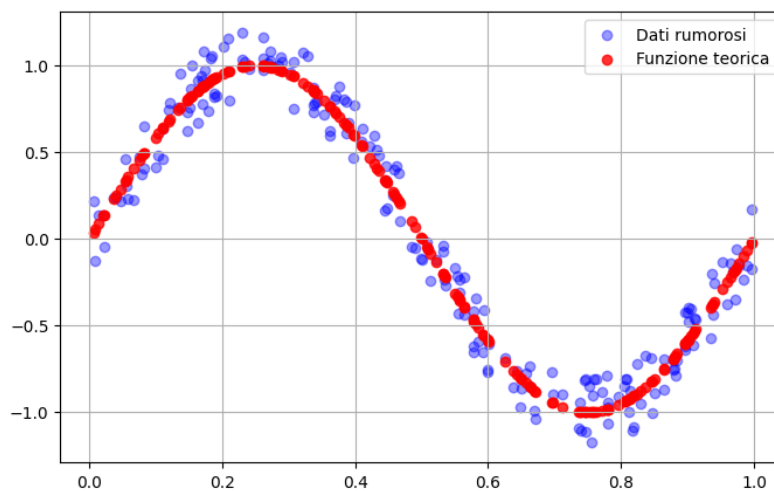


Figura 1: Dati - in blue- generati a partire da $\sin(2\pi x)$ con l'aggiunta di un rumore uniforme. In rosso la funzione $\sin(2\pi x)$.

2 Modello Polinomiale

```
1 def pol(n, X=None):
2     if X is None:
3         X = np.linspace(0, 1, 100) # default: 100 punti fra 0 e 1
4
5     # Coefficienti casuali tra -0.5 e 0.5
6     coef = np.random.uniform(-0.5, 0.5, size=n+1)
7     p = np.polynomial.Polynomial(coef)
8     Z = p(X)
9
10    return Z, coef
```

Si riporta una prova del metodo per creare vari polinomi:

```

1 n = np.linspace(1,4,4)
2 for i in n:
3     i = int(i)
4     X = np.linspace(-3,3, 10000)
5     Z, _ = pol(i,X)
6
7     plt.scatter(X, Z, s=10, color='red', alpha=0.3, label=f'Polinomio di {i} grado')
8     plt.grid(True)
9     plt.legend()
10    plt.show()

```

Si riportano i risultati grafici [Fig.2]:

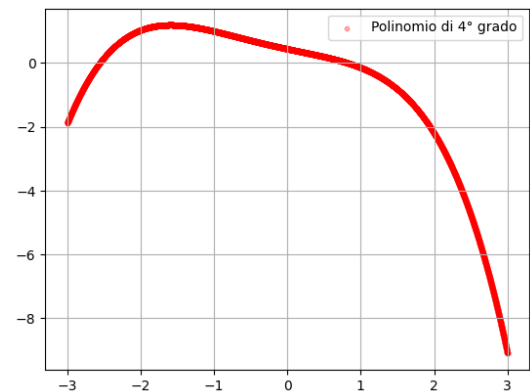
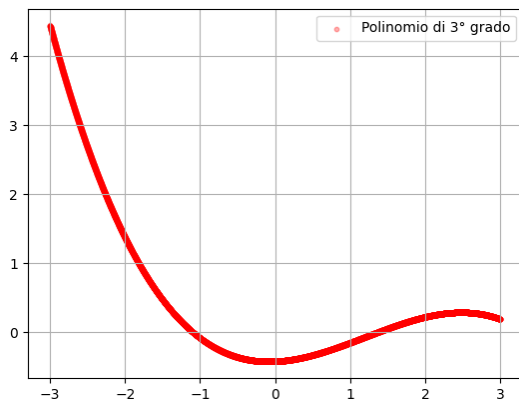
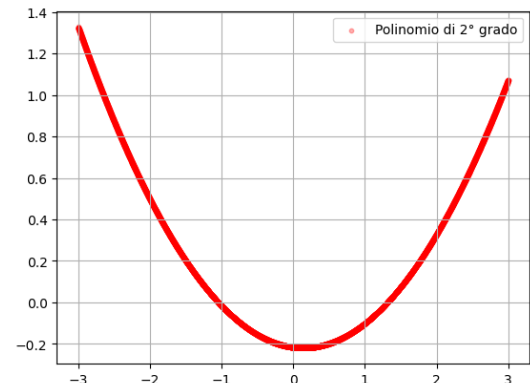
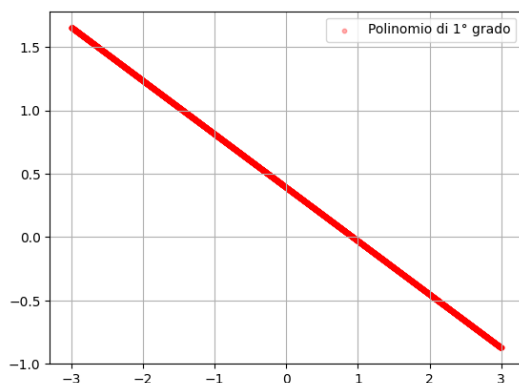


Figura 2: Polinomi di grado 1, 2, 3 e 4 in $[-3,3]$

Come ci si attende aumentare il grado modifica il numero di zeri del polinomio. Inoltre modifica la sua curvatura: aumentando il grado aumentano il numero di derivate che possiamo svolgere sul polinomio prima di ottenere il risultato nullo.

3 Discesa del Gradiente

Definite le funzioni costo e gradiente:

```

1 def J(omega, Y, X, N):
2     p = np.polynomial.Polynomial(omega)
3     elements = (Y - p(X))**2
4     return np.sum(elements)/(2*N)
5
6
7 def grad_J(omega, Y, X, N):
8     M = len(omega)
9     grad = np.zeros(M)
10    error = (Y - np.polynomial.Polynomial(omega)(X))
11    for j in range(M):

```

```

12     # derivata rispetto a omega_j
13     grad[j] = np.sum(error * X**j)
14     grad[j] = -grad[j] / N
15
16     return grad

```

Si è implementata la discesa del gradiente per un modello polinomiale di grado $n = 10$ iterata per un numero di $steps = 80000$ con vari valori del tasso di apprendimento $\eta = [0.1; 0.01; 0.001]$:

```

1  etas = [0.1, 0.01, 0.001]
2  steps = 80000
3  n = 10
4
5  results = {} # per salvare (omega_finale, J_history) per ogni eta
6
7  # Calcolo una sola volta per ogni eta
8  for eta in etas:
9      p, omega = pol(n, X)
10     J_history = []
11
12     for i in range(steps + 1):
13         omega = omega - eta * grad_J(omega, Y, X, N)
14         cost = J(omega, Y, X, N)
15         J_history.append(cost)
16
17     results[eta] = (omega, J_history)
18
19 #Dati e polinomi
20 plt.figure()
21 plt.scatter(X, Y, color='red', alpha=0.4, label='Dati')
22
23 for eta, (omega, _) in results.items():
24     plt.scatter(X, np.polynomial.Polynomial(omega)(X), alpha=0.6, label=f'Polinomio_eta={eta}')
25
26 plt.legend()
27 plt.title('Dati e polinomi')
28 plt.grid(True)
29 plt.show()
30
31 # Funzione di costo
32 fig, ax = plt.subplots()
33
34 for eta, (_, J_history) in results.items():
35     ax.plot(range(steps + 1), J_history, label=f'eta={eta}')
36
37 ax.set_xlabel('Step')
38 ax.set_ylabel('J(omega) - Funzione di costo')
39 ax.set_title('Evoluzione della funzione di costo')
40 ax.grid(True)
41 ax.legend()
42 plt.show()

```

Si riportano i risultati grafici [Fig.3]:

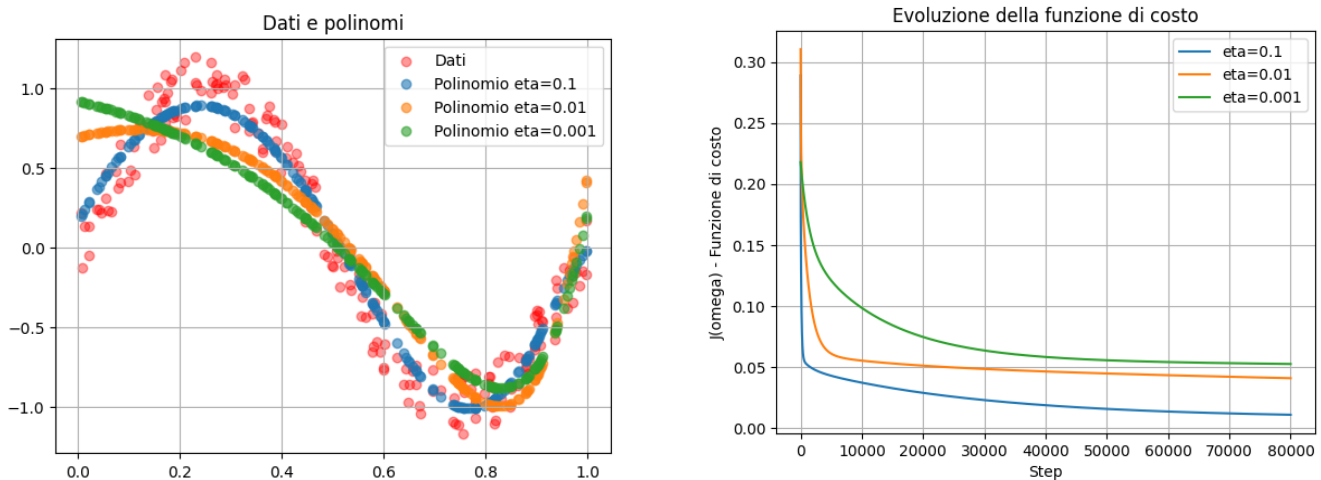


Figura 3: La figura a sinistra mostra i polinomi ottenuti dopo la discesa del gradiente, confrontandoli con i dati (in rosso). La figura a destra mostra l'andamento delle funzioni di costo durante la discesa del gradiente. In entrambi i grafici il blu indica un η di 0.1, arancione di 0.01, verde di 0.001.

Il tasso di apprendimento di $\eta = 0.1$ ha raggiunto un miglior risultato nell'apprendere i dati.

4 Monitorare la Convergenza

Si è considerato un unico valore di $\eta = 0.1$ e ne si è monitorata la convergenza.

```
1 eta=0.1
2 steps=80000
3 n=10
4 p, omega = pol(n,X)
5 J_history = []
6
7 for i in range (steps+1):
8     omega = omega - eta*grad_J(omega,Y,X,N)
9     cost = J(omega, Y, X, N)
10    J_history.append(cost)
11    #Grafico Dati - Polinomio
12    if (i % 8000 == 0 or i%80000 == 1):
13        plt.scatter(X, Y, color='red',alpha=0.4, label='Dati')
14        plt.scatter(X, np.polynomial.Polynomial(omega)(X),color='blue', label=f'Iterazione_{i}')
15        plt.legend()
16        plt.grid(True)
17        plt.show()
18
19 # Grafico della funzione di costo
20 plt.scatter(range(steps+1), J_history)
21 plt.xlabel('Step')
22 plt.ylabel('J(omega)')
23 plt.title('Evoluzione_della_funzione_di_costo')
24 plt.grid(True)
25 plt.show()
26
27 print(omega)
```

Si riportano i risultati grafici [Fig.4] e [Fig.5]:

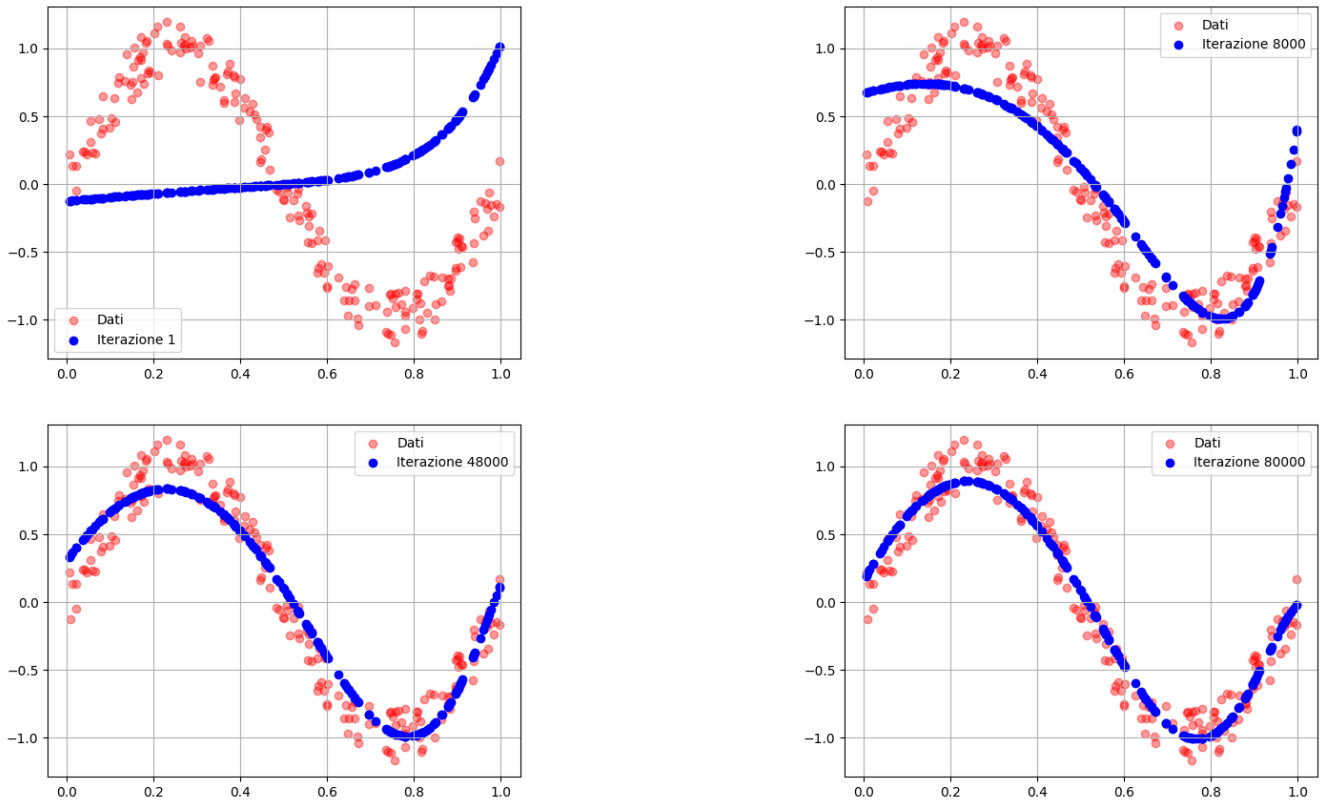


Figura 4: Varie iterazioni dell'implementazione della discesa del gradiente con $\eta = 0.1$. In rosso i dati, in blu il polinomio ottenuto in quella iterazione.

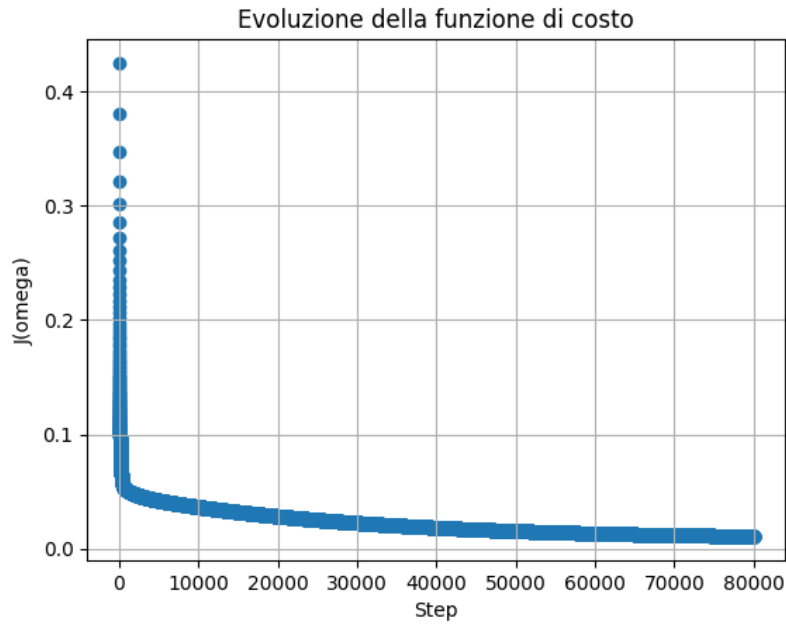


Figura 5: In blu l'andamento della funzione costo durante le iterazioni a $\eta = 0.1$.

5 Mini-Batch SGD

Si è implementato in primo luogo una funzione in grado di creare i batches:

```

1 def create_batches(X, Y, batch_size):
2     N_train = len(X)
3     X_batches = []
4     Y_batches = []
5
6     for i in range(0, N_train, batch_size):
7         X_batches.append(X[i:i + batch_size])
8         Y_batches.append(Y[i:i + batch_size])
9
10    return X_batches, Y_batches

```

Si è suddiviso il dataset in Train e Test e si riporta graficamente tale suddivisione [Fig.6]:

```

1 test_ratio = 0.2
2 N_test = int(N * test_ratio)
3 N_train = N - N_test
4 indices = np.arange(N)
5 np.random.shuffle(indices)
6 X_train, Y_train = X[indices[N_test:]], Y[indices[N_test:]]
7 X_test, Y_test = X[indices[:N_test]], Y[indices[:N_test]]
8 #Plot Dati
9 plt.scatter(X_train, Y_train, color='blue', label='Dati Train')
10 plt.scatter(X_test, Y_test, color='green', label='Dati Test')
11 plt.legend()
12 plt.grid(True)
13 plt.show()

```

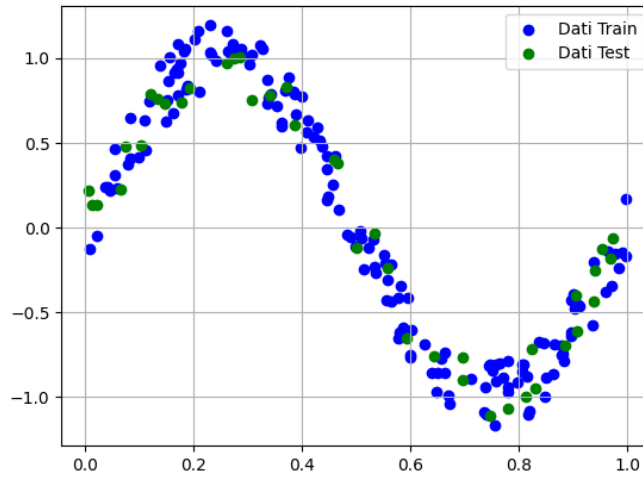


Figura 6: Grafico dei dati di partenza ora suddivisi in due gruppi. In blu i dati di train, in verde i dati di test.

Successivamente si è implementata la Mini-Batch SGD con varie dimensioni dei batches, ma ad $\eta = 0.1$, grado del polinomio $n = 10$ e $steps = 40000$

```

1 eta = 0.1
2 steps = 40000
3 n=10
4 p, omega0 = pol(n,X)
5 results = {}
6
7 batch_sizes = [1, 10, 20, 40, 80, 160]
8
9 for batch_size in batch_sizes:
10     #partire dalle stessa situazione iniziale
11     J_history=[]
12     omega=omega0
13     num_batches = int(N_train / batch_size)
14
15     for i in range(steps+1):
16         if (i%(num_batches)==0):
17             indices = np.arange(N_train)
18             np.random.shuffle(indices)
19             X_train = X_train[indices]
20             Y_train = Y_train[indices]
21             X_batches, Y_batches = create_batches(X_train, Y_train, batch_size)
22             index_batches=i%num_batches
23             current_batch_size = len(X_batches[index_batches])
24             omega = omega - eta * grad_J(omega,Y_batches[index_batches],X_batches[index_batches],current_batch_size)
25             cost = J(omega, Y_train, X_train, N_train) #calcolato sui dati di train
26             J_history.append(cost)
27
28     results[batch_size] = J_history

```

Per studiare come influisca la dimensione dei batches sulla velocità di convergenza, stabilità e rumorosità del metodo si mette in evidenza come cambi la funzione costo.

```

1 #Grafico delle funzioni costo in scala log
2 plt.figure(figsize=(10,6))
3
4 for batch_size, J_history in results.items():
5     plt.plot(range(len(J_history)), J_history, label=f'batch={batch_size}', linewidth=1)
6
7 plt.yscale('log')
8 plt.xlabel('Step')
9 plt.ylabel('J() Funzione di costo in scala logaritmica')
10 plt.title('Confronto delle funzioni di costo per diversi batch_size')
11 plt.grid(True, which='both')
12 plt.legend()
13 plt.show()
14
15 #Grafico: Normalizzazione per epoche
16 plt.figure(figsize=(10,6))
17 for batch_size, J_history in results.items():
18     # ogni epoca = passaggio completo sui N_train
19     num_batches = int(N_train / batch_size)
20     #rinormalizzo il vettore delle iterazione per il numero di batches
21     epochs = np.arange(len(J_history)) / num_batches
22     plt.plot(epochs, J_history, label=f'batch={batch_size}', linewidth=1)
23 plt.xlabel('Epoche')

```

```

24 plt.ylabel('J() - Funzione di costo')
25 plt.title('Funzione di costo normalizzata per epoche')
26 plt.grid(True, which="both", alpha=0.5)
27 plt.legend()
28 plt.show()

```

Risultati grafici:

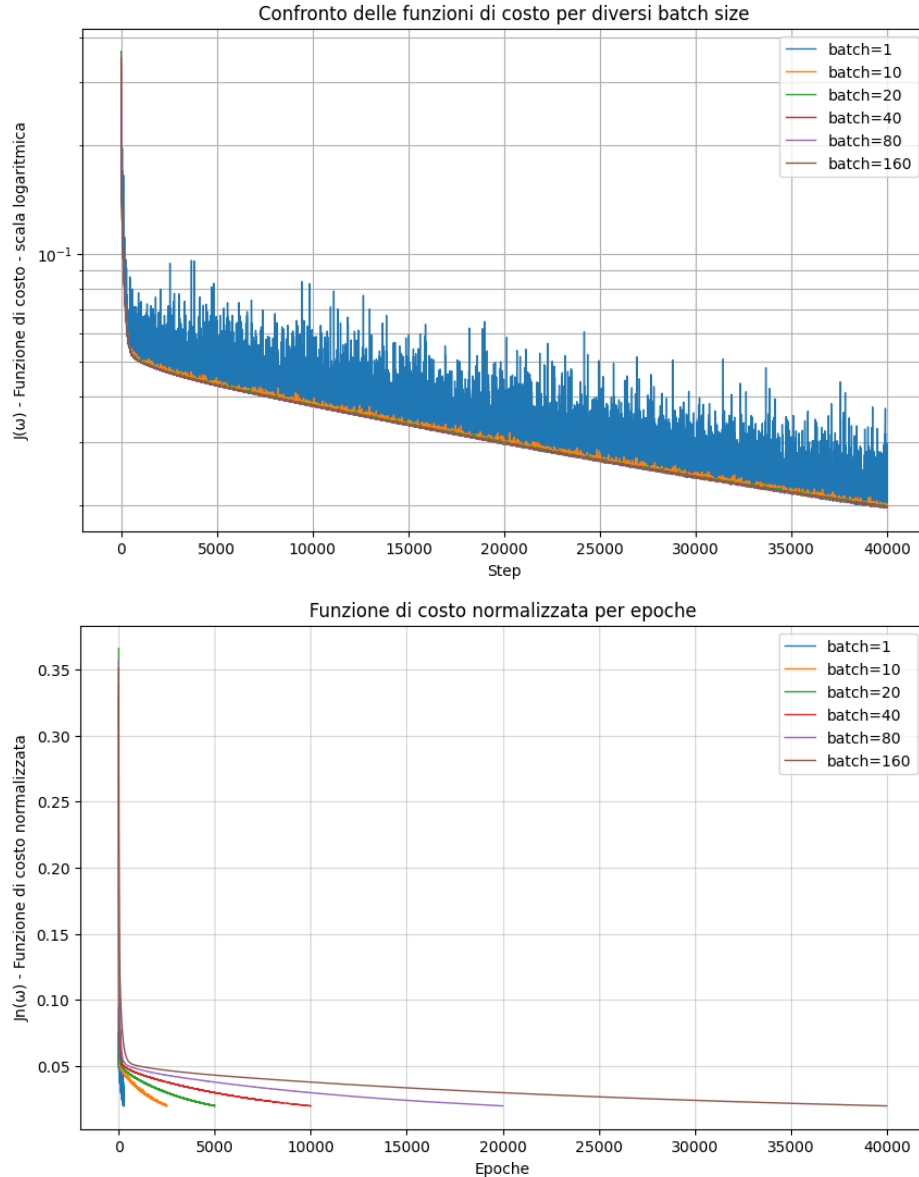


Figura 7: Il primo grafico mostra l'andamento della funzione di costo in scala logaritmica per varie dimensioni dei batch. Il secondo grafico mostra la funzione di costo normalizzata per epoche.

6 Riflessioni Finali

I grafici di [Fig.7] ci permettono di fare alcune valutazioni. Se la dimensione di ogni batch è molto piccola la funzione costo è molto rumorosa, mentre per dimensioni sempre maggiori essa diventa sempre più liscia. Questo permette di comprendere come la funzione costo durante le iterazioni sia più stabile nel caso di grandi batch. Il secondo grafico invece ci permette di vedere come

una dimensione piccola dei batch permetta di velocizzare la discesa del gradiente, ovvero in meno iterazioni riesce a far diminuire il valore della funzione di costo. Questo a discapito della sua stabilità come detto precedentemente.

6.1 Riflessioni sulla Complessità del modello

Durante il laboratorio ho sempre utilizzato $n = 10$ come grado del polinomio del modello. Si mostra un confronto fra errore commesso e complessità del modello considerando $steps = 40000$ e facendo variare il grado del modello monitorando l'ultimo valore della funzione $J(\omega)$

```

1 eta = 0.1
2 steps = 40000
3 ns = [1,2,3,4,5,6,7,8,9,10,11,12,13,15,20,25,30,35,40,50,70,100]
4 J_last_train=[]
5 J_last_test=[]
6
7 for n in ns:
8     p, omega = pol(n, X)
9     for i in range(steps + 1):
10         omega = omega - eta * grad_J(omega, Y_train, X_train, N_train)
11
12     # Calcola errore su entrambi
13     cost_train = J(omega, Y_train, X_train, N_train)
14     cost_test = J(omega, Y_test, X_test, N_test)
15     if(i==steps):
16         J_last_train.append(cost_train)
17         J_last_test.append(cost_test)
18     print(f"fatto grado {n}")

```

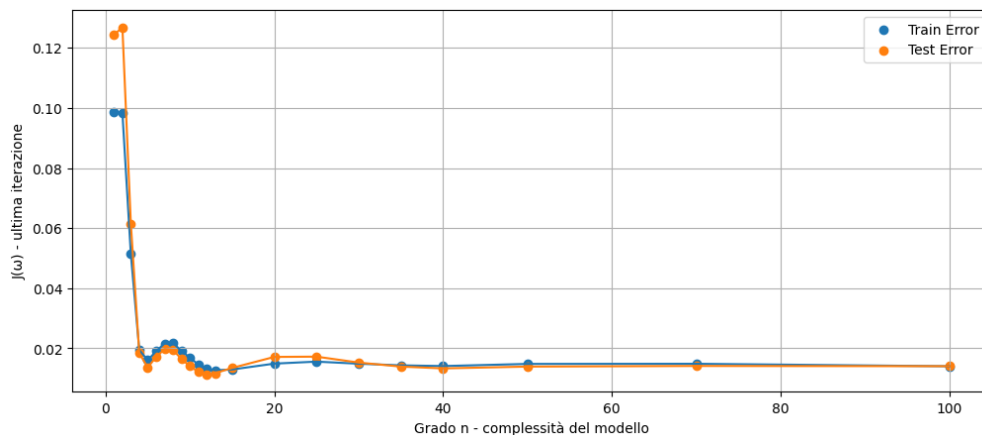


Figura 8: Il risultato grafico dello studio fra costo all'ultima iterazione e grado del polinomio usato come modello. In blu il calcolo del costo sui dati di Train, mentre in arancione sui dati di Test.

Il grafico presenta fino ad un certo punto il classico grafico di bias-variance tradeoff: i primi modelli con grado troppo piccolo presentano un errore maggiore, poichè si tratta di una regione di underfit. Raggiunto il minimo locale a $n = 5$ l'errore inizia ad aumentare, poichè si tratta di una regione di overfit. Successivamente si nota un nuovo calo nell'errore. Qualitativamente per il polinomio di grado $n = 10$ la funzione costo non ha grandi differenze fra errore di test e train, ma sicuramente esso non è un minimo, quindi sarebbe stato possibile trovare un modello migliore. La seconda discesa inaspettata presente dopo il primo picco in $n = 7$ raggiunge errori minori rispetto a quelli in $n = 5$, mostrando che alcuni modelli più complessi non avrebbero avuto un overfitting.